HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

Đề tài: Finetune mô hình dịch máy Neural cho văn bản chuyên ngành y khoa Anh-Việt

Lớp tín chỉ:	**D23CQCN04-B**
Giảng viên hướng dẫn:	**TS. Kim Ngọc Bách**
Sinh viên thực hiện:	**Nguyễn Đức Trung**
Mã sinh viên:	**B23DCCN858**

Hà Nội 2026

MỤC LỤC

I. MỤC TIÊU CỦA TUẦN LÀM VIỆC 2

1. **Mục tiêu** ..... Error! Bookmark not defined.
2. **Nội dung đã thực hiện** ..... Error! Bookmark not defined.
3. **Kết quả đạt được** ..... Error! Bookmark not defined.
4. **Kế hoạch tuần tới** ..... Error! Bookmark not defined.

I. MỤC TIÊU CỦA TUẦN LÀM VIỆC

1. Mục tiêu

- Xây dựng một chatbot chuyên dịch văn bản trong lĩnh vực y tế (Anh → Việt) dựa trên mô hình **T5** đã được finetune (Envit5 + MedEV).
- Triển khai mô hình dưới dạng một **dịch vụ API** để có thể tích hợp với frontend (web/app), hướng tới ứng dụng thực tế như:
    - hỗ trợ dịch tài liệu y khoa
- Tối ưu tốc độ suy luận để đạt độ trễ thấp, phù hợp với trải nghiệm người dùng.

2. Nội dung đã thực hiện

2.1. Chuẩn bị mô hình

- Hoàn thành finetune mô hình Envit5 trên tập dữ liệu y tế (MedEV).

2.2. Nghiên cứu hướng triển khai

Đã tìm hiểu nhiều hướng deploy khác nhau:

a. Deploy bằng API tự xây dựng

- Sử dụng **FastAPI** để xây dựng API:
    - endpoint /translate nhận input text
    - trả về kết quả dịch từ model
- Đây là hướng cơ bản, dễ kiểm soát và phù hợp để tích hợp với hệ thống Django hiện tại.

b. Sử dụng inference server có sẵn

- Tìm hiểu **Text Generation Inference**:
    - Có khả năng load trực tiếp model và tự tạo API

    - Hỗ trợ batching và tối ưu GPU
- Tuy nhiên nhận thấy:
    - TGI tối ưu tốt hơn cho mô hình dạng GPT (decoder-only)
    - Với T5 (encoder-decoder), hiệu năng chưa tối ưu hoàn toàn

c. Tối ưu tốc độ bằng ONNX

- Áp dụng **ONNX Runtime** để tăng tốc inference
- Sử dụng **Optimum** để:
    - convert model sang ONNX
    - chạy inference nhanh hơn so với PyTorch
- Đây là hướng được đánh giá phù hợp nhất với mô hình T5

d. Thử nghiệm môi trường GPU miễn phí

- Sử dụng Kaggle (GPU T4) để:
    - test inference
    - benchmark tốc độ
- Nhận thấy:
    - môi trường này không phù hợp để deploy server lâu dài, nó chỉ deploy local được, không thể call từ một phía khác được.

e. Khảo sát các nền tảng deploy

Đã nghiên cứu các lựa chọn:

- Render / Railway
  → miễn phí, có server nhưng chỉ CPU
- Hugging Face
  → có thể deploy nhanh (Spaces / API)
- RunPod
  → có GPU, phù hợp production nhưng không miễn phí

3. Kế hoạch tuần tới

3.1. Hoàn thiện pipeline deploy

- Chuyển hoàn toàn model sang ONNX
- Xây dựng API hoàn chỉnh bằng FastAPI:
    - xử lý input/output rõ ràng

    - chuẩn hóa format prompt cho T5

3.2. Deploy thử nghiệm

- Triển khai server trên nền tảng miễn phí (Render hoặc Railway)
- Kiểm tra:
    - thời gian phản hồi (latency)
    - độ ổn định

3.3. Tích hợp với frontend

- Kết nối API với hệ thống Django hiện có
- Xây dựng giao diện chatbot cơ bản:
    - nhập văn bản
    - hiển thị kết quả dịch

4. Khó khăn và thách thức

4.1. Hiệu năng inference

- Mô hình T5 có kiến trúc encoder–decoder → tính toán nặng hơn GPT
- Khi chạy mặc định (PyTorch, CPU):
    - tốc độ rất chậm
- Cần nhiều bước tối ưu (ONNX, GPU, decoding strategy)

4.2. Hạn chế của môi trường miễn phí

- Kaggle:
    - không hỗ trợ server lâu dài
    - không expose API public ổn định
- Render/Railway:
    - không có GPU
    - dễ bị sleep

4.3. Độ phức tạp khi deploy

- Khác biệt lớn giữa:
    - môi trường nghiên cứu (notebook)
    - môi trường production (server, API, scaling)
- Cần xử lý thêm:

    - batching
    - quản lý request
    - tối ưu tài nguyên

4.4. Giới hạn của các công cụ có sẵn

- TGI không tối ưu hoàn toàn cho T5
- Một số tool phổ biến hiện nay tập trung vào LLM (GPT-style), không phù hợp hoàn toàn với bài toán dịch.